SOLUTION STATEMENT

Micron AI-Aware Cache & Prefetch Engine RTL Challenge

A confidence-gated, accuracy-adaptive stride prefetcher for a 4 KB set-associative cache

Submitted by	Kotapati Kishore (Roll No. EE26E016), B.Tech EEE, NIT Manipur; final year at IIT Madras
Date	1 November 2026
Repository	https://github.com/Kishore-Kotapati/micron_cache_prefetch
Headline	2.26x average speedup over the same cache without prefetching (9 workloads), 2.06x on AI-inspired workloads, never slower; 3.03x with 120-cycle DRAM

In one sentence: a small, self-tuning stride prefetcher - under 5 % of the hardware - makes the same 4 KB cache 2.26x faster on average (2.06x on AI-inspired workloads, 3.03x with slower memory), verified on two simulators against a golden model.

1. Problem

AI workloads such as large language models, CNNs and recommendation systems are fast at arithmetic but waste time waiting for data from DRAM. The challenge: build hardware that **watches** the addresses a program reads, **guesses** the next ones and **fetches them early** into the cache, and prove that it is faster (higher hit rate, lower latency, good prefetch accuracy) using Python, Icarus Verilog, GTKWave and Git, with no starter code.

2. Proposed solution

A complete cache subsystem written from scratch in synthesizable Verilog-2005:

- **Cache:** 4 KB, 4-way set-associative, 64 sets, 16-byte lines, true LRU, 1-cycle hit, 8 miss-status registers (MSHRs) so up to 7 prefetches can travel while the CPU keeps working.
- **Prefetch engine (watch - guess - fetch):** a 4-entry stream table learns the byte stride of up to four interleaved streams (no program counter needed), with a 2-bit confidence counter; when confident it generates the next lines, never across a 4 KB page, removes duplicates, and sends them to DRAM only in cycles the CPU does not need.
- **Correctness rules:** demand requests always win the DRAM port; one MSHR is always reserved for the CPU; one cache update per cycle; a load to a line already in flight merges with it instead of fetching it twice.

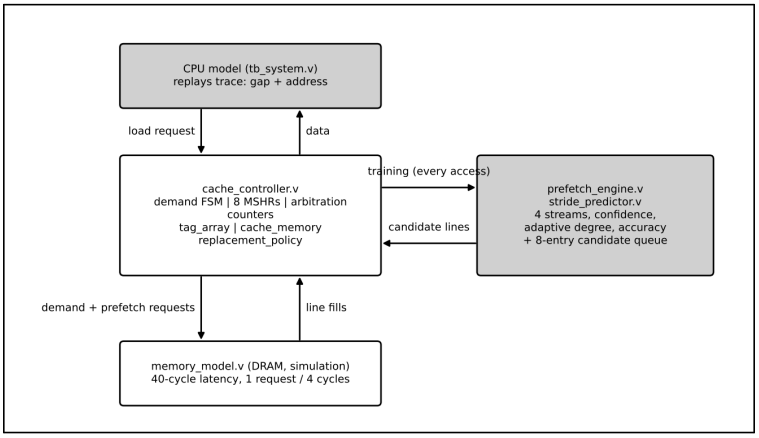


Figure 1: Solution architecture

3. What makes it intelligent (innovation)

One small accuracy monitor (two 6-bit counters) measures, every 32 prefetches, how many were used, and drives three adaptive mechanisms - one per innovation area on the official slide:

Table 1: Innovations and their measured contribution

Innovation	Mechanism	Measured effect
Intelligent prefetching	degree (lines ahead) rises to 7 when >= 75 % of prefetches are used, falls when < 25 %	average 1.84x -> 2.21x
Novel prediction / AI-aware	a new or jumping stream that misses gets 2 next-line prefetches at once; off when accuracy < 50 %	CNN 2.00x -> 2.18x; random traffic wastes only 32 prefetches
Adaptive cache policy	prefetched lines inserted as least-recently-used while accuracy < 50 %, else most-recently-used	recsys 1.37x -> 1.46x with 9 % less DRAM traffic

4. Results

Architecture A (no prefetch) and B (final design) use identical RTL, traces and DRAM timing (40-cycle latency, one request per 4 cycles). All values come from RTL simulation logs.

Table 2: Baseline (A) vs final design (B)

Workload	Hit % A -> B	Avg latency A -> B (cyc)	Cycles A -> B	Speedup	Prefetch accuracy*
sequential	75.0 -> 99.4	11.25 -> 1.07	112571 -> 29172	**3.86x**	100 %
stride	0.0 -> 21.1	42.00 -> 6.04	133557 -> 25688	**5.20x**	100 %
random	0.3 -> 0.3	41.89 -> 41.89	177688 -> 177688	**1.00x**	0 %
repeated	97.5 -> 98.3	2.02 -> 1.71	17357 -> 16127	**1.08x**	94 %
mixed	34.5 -> 72.8	27.83 -> 11.89	182100 -> 86455	**2.11x**	58 %
capacity	75.0 -> 99.1	11.25 -> 1.09	84464 -> 22016	**3.84x**	100 %
cnn	80.2 -> 95.8	9.13 -> 2.02	112549 -> 51680	**2.18x**	67 %
llm	84.4 -> 99.5	7.41 -> 1.10	82323 -> 29855	**2.76x**	97 %
recsys	88.3 -> 93.8	5.78 -> 3.17	79479 -> 54379	**1.46x**	47 %
average				**2.26x**	

**Accuracy = (useful + late prefetches) / prefetches issued.* Regular streams (sequential, capacity, LLM) reach 98-99 % of a perfect cache; random addresses cannot be predicted (1.00x, correctly almost no prefetches); repeated data already fits in the cache.

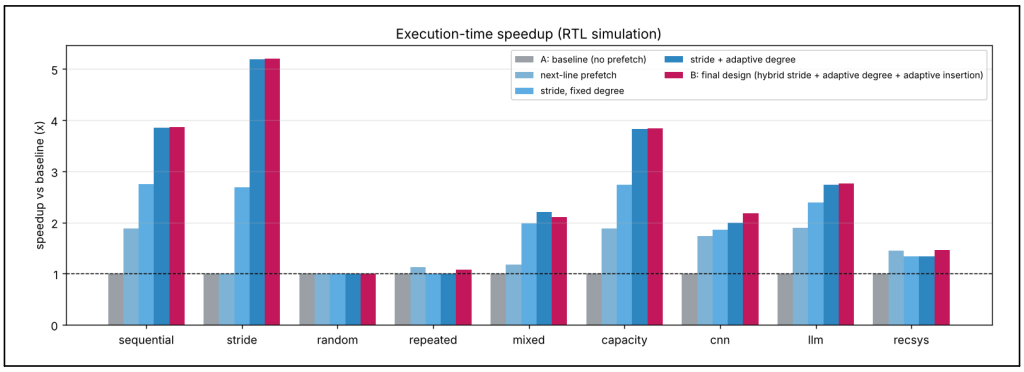


Figure 2: Speedup of each architecture step (higher is better)

Table 3: The benefit grows as memory gets slower

DRAM latency (cycles)	20	40	80	120
Average speedup, 9 workloads	1.82x	2.26x	2.82x	3.03x
AI-inspired workloads	1.62x	2.06x	2.65x	2.97x

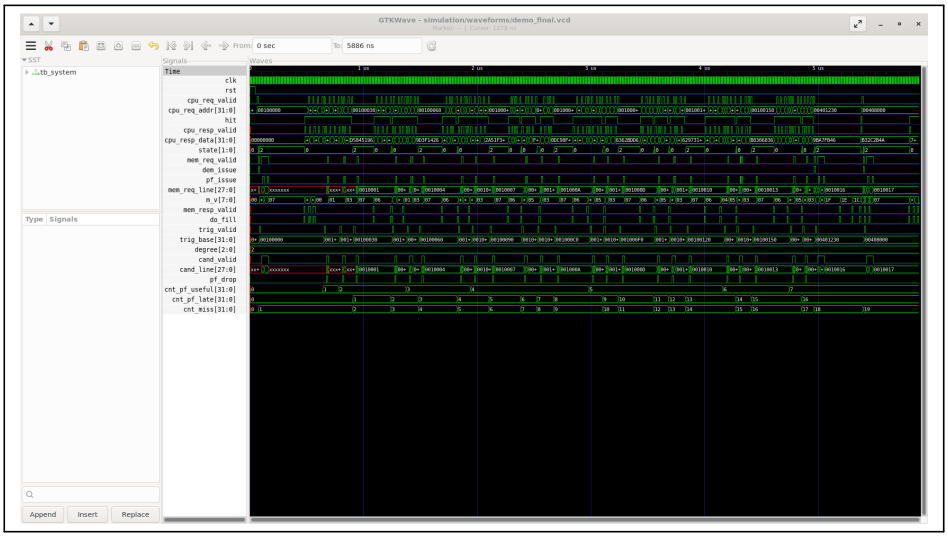


Figure 3: GTKWave, final design: 98 loads in 5,886 ns (baseline: 13,816 ns); only 3 demand DRAM requests (baseline: 26)

5. How we proved it works

Table 4: Verification summary

Check	Result
Self-checks in every simulation (data, counters, no duplicates, prefetch accounting, drain)	54 / 54 PASS
Baseline vs independent Python golden cache model	exact match on 9 / 9 workloads
Second simulator (Verilator 5.020) running the same testbench	54 / 54 runs bit-identical to Icarus
Cocotb Python tests (latency, LRU, 1500 random loads, merge, page boundary, throttling)	7 / 7 PASS
Verilator lint	0 warnings
Yosys synthesis (generic gates)	clean; prefetch engine = 4.9 % of cells, 1.8 % of flip-flops
Reproduced on a second machine (Ubuntu 24.04, WSL)	identical numbers

6. Limitations

Synthetic traces (models of AI access patterns, not recordings of real programs); read-only, single-level cache with an in-order CPU model; fixed-latency DRAM; generic-gate synthesis only (no area, timing or power claimed). Prefetching costs extra DRAM traffic on CNN, recsys and mixed (1.4-1.8x). All are documented in the technical report.

7. Deliverables and reproduction

- Source, results, waveforms and documentation: **https://github.com/Kishore-Kotapati/micron_cache_prefetch**
- Technical report: docs/Micron_AI_Aware_Cache_Prefetch_Engine_Technical_Report.pdf
- One command reproduces everything in about one minute: `bash run_all.sh`